



Resumen Jeff Patton — Product, Agile y por qué el proceso importa

El mundo cambió: ya no hacemos software para entregar en cajas, sino productos que mejoran continuamente. Por eso, las formas de trabajar deben girar en torno a impacto real, no a entregas a tiempo.

Nadie define un producto grandioso como «se entregó en fecha y dentro del presupuesto». Lo definen como: resuelve un problema, la gente lo usa, da valor, es intuitivo, cambia vidas.

1. El modelo Output → Outcome → Impact

Output	Lo que construye el equipo: features, código. Controlado 100% por el equipo.
Outcome	Cambio en el comportamiento de los usuarios: ven el producto, lo prueban, lo usan y lo siguen usando. Lo controlan los usuarios, NO el equipo.
Impact	Resultado de negocio: revenue, market share, NPS, ROI. Consecuencia del outcome, no del output.



Clave: un producto es grandioso solo si genera outcome e impact. Medir el éxito solo por output (entregar a tiempo) es una trampa.

2. Los 3 grandes problemas con las ideas

Problema 1 — Hay demasiadas

Clientes, stakeholders y el equipo generan ideas constantemente. A más éxito tiene la empresa, más ideas entran al sistema.

Problema 2 — La mayoría son malas

92% de las startups fracasan (Startup Genome Report). El 65-75% de las features raramente o nunca se usan (Chaos Report, Standish Group). Marty Cagan (ex-eBay): si sos realmente bueno, acertás 1 de cada 3 veces. El 50-80% del software falla en cumplir sus objetivos.

Problema 3 — El sesgo del optimismo

Todos saben que el 90% de las startups fracasan... pero no la propia. Nuestro cerebro imagina el futuro en «future perfect»: todo va a funcionar. Este sesgo nos hace ignorar la evidencia y construir cosas que nadie usa.

3. Historia del desarrollo de software

80s – 90s	Software en caja (floppy disks). Requisitos fijos. Triángulo tiempo/costo/scope: fijar dos de los tres explotaba la calidad o el tercero.
1986	Paper de Takeuchi & Nonaka en HBR: el desarrollo exitoso es caótico pero ordenado, como el rugby. Usan la palabra 'scrum'. Origen del nombre Scrum.
2001 — Ágil	Manifiesto Ágil: software funcionando > documentación, responder al cambio, equipos cross-funcionales, entregas pequeñas y frecuentes.
2001 — Design	Frank Robinson acuña MVP = Minimum Viable Product = el release más pequeño que puede ser exitoso en el mercado (no un test).
2011	Eric Ries (Lean Startup) redefine MVP como 'next best test'. DevOps escala. Marc Andreessen: 'software is eating the world'. Spotify model.
Hoy	Todo es software: relojes, autos, lamparitas, motores a reacción (Rolls-Royce vende 'power by the hour'). Los productos mejoran continuamente.

4. El MVP y su confusión histórica

Hay tres definiciones de MVP que son completamente distintas entre sí — y dos de ellas son opuestas:

MVP original	Frank Robinson (2001)	El release más pequeño que puede ser EXITOSO: genera ROI real, se queda en el mercado
MVP Lean Startup	Eric Ries (2011)	El TEST más pequeño para validar una hipótesis — no necesariamente shippeable a todos
MVP en la mayoría	Nadie en particular	Lo máximo que entra antes de la fecha = primer release horrible

Recomendación: dejar de usar el término MVP y ser explícito:

«Smallest successful release» — para un release mínimo que genere ROI real.

«Next best test» — para un experimento que reduce incertidumbre.

«First crappy release» — si eso es lo que realmente están haciendo. Al menos sean honestos

5. Build – Measure – Learn (Eric Ries)

El ciclo de validated learning parte de la idea de que la mayoría de los supuestos son riesgosos y hay que testarlos antes de invertir en construir a escala.

Idea / hipótesis	El punto de partida. El backlog no es de features sino de riesgos y supuestos.
Suposición más riesgosa	¿La gente va a querer esto? ¿Pueden usarlo? ¿Podemos construirlo a tiempo? La más importante va al tope del backlog.
Construir el test mínimo	Puede ser un prototipo, una landing page, un press release — no necesariamente software shippeable.
Medir / observar	El objetivo no es ROI ni clientes felices: es datos que reducen incertidumbre.
Aprender	Los datos cambian los supuestos y revelan la próxima suposición más riesgosa. El ciclo se repite.

Crítica al naming de Ries: «Build» hace pensar en software terminado, «Measure» en analytics de producción. Los nombres son malos pero democratizó ideas

fundamentales. El ciclo puede completarse en horas o días.

6. El modelo de Spotify (Henrik Kniberg)

1. Investigación: ¿la gente quiere esto? ¿resuelve un problema real?
2. Narrativa: press release o elevator pitch con los beneficios.
3. Hipótesis: ¿cómo va a cambiar el comportamiento de los usuarios?
4. Prototipos y pruebas con usuarios reales.
5. MVP shippeable = lo mínimo para cumplir la narrativa, lejos de estar completo
→ se lanza al 2-3% de usuarios.
6. A/B testing y monitoreo de métricas de impacto.
7. Si funciona → rollout gradual. Si no → no se lanza a todos

Lo que se deja afuera del MVP inicial: funcionalidad de escala (localización, performance para millones), features no esenciales, training de soporte. Solo lo mínimo para que el experimento gire.



«Un feature no está done hasta que alcanza el impacto deseado.»

7. «Software is eating the world» (Andreessen, 2011)

Todo se convirtió en software: teléfonos, relojes, termostatos, autos (Tesla recibe updates diarios), motores a reacción (Rolls-Royce vende 'power by the hour' y los monitorea en tiempo real), juegos (Fortnite iteró al éxito), entretenimiento (Netflix y Amazon > Hollywood). Los bancos compiten con fintechs.

La consecuencia: ya no trabajamos en iteraciones cortas para mostrar progreso a stakeholders.

Trabajamos en iteraciones cortas para que nuestra empresa no muera.

8. El proceso continuo moderno

1. Sensar / aprender	Datos del producto en producción + conversaciones con usuarios. Producto en el mercado → retroalimentación constante.
2. Generar opciones	Ideas, features, mejoras — sabiendo que la mayoría no va a funcionar. No hay que construirlas todas.
3. Siguiendo test	Build-Measure-Learn antes de shippear a todos. Reducir riesgo con evidencia.
4. Escalar solo si funciona	Los sprints ágiles viven aquí: construir de a poco hasta tener confianza. Nunca rollout masivo de algo no validado.

Amazon shippea cambios cada 11.6 segundos (dato de 2011) — pero no features completas a todos. Son pequeños cambios a un pequeño porcentaje de usuarios, iterando hasta que funcionan.

9. Story Mapping (Q&A;)

Mapa del presente	Cómo usan el producto los usuarios HOY, paso a paso.
Identificar problemas	Dónde duele y dónde es bueno el producto actual.
Evaluar el backlog	¿Las ideas actuales arreglan lo que duele o amplifican lo bueno?
Mapa del futuro	Imaginar cómo sería la experiencia si se construye lo nuevo.

Smallest release	Usar el mapa para encontrar el release mínimo exitoso — no 'lo máximo que entra', sino 'lo mínimo que funciona'.
-------------------------	--

10. Tabla resumen — conceptos clave

Output	Lo que construye el equipo (features, código)
Outcome	Cambio en comportamiento de usuarios (fuera del control del equipo)
Impact	Resultado de negocio: revenue, NPS, market share
MVP (Robinson)	Smallest successful release — genera ROI real
MVP (Ries)	Next best test — mínimo para aprender
Validated learning	Reducir riesgo con evidencia antes de escalar
Optimism bias	Creer que nuestra idea es la excepción al fracaso
Build to learn	Testear ≠ lanzar a todos. El objetivo es datos, no revenue.
Scrum (origen)	Del paper Takeuchi/Nonaka 1986 — no nació en software
Software eating world	Todo es software, todo mejora continuamente



«Un buen proceso te sirve para poder servir a tus clientes. Pero si no estás atento, el proceso puede convertirse en la cosa en sí.»